

useResponsiveLayout.js — Fundamentos

Explicación detallada del custom hook de layout responsive (GymBro App)

Es un **custom hook**. Es un archivo chico (22 líneas) pero conceptualmente importante porque es el hook que **todas** las pantallas de la app usan para adaptarse a rotación de pantalla y distintos tamaños de dispositivo.

1. Qué es un “custom hook” (a diferencia de un componente)

```
import { useWindowDimensions } from 'react-native';

export function useResponsiveLayout() {
  const { width, height } = useWindowDimensions();

  return {
    width,
    height,
    isLandscape: width > height,
    isWideScreen: width >= 700,
  };
}
```

Un componente (como PrimaryButton o CustomInput) **devuelve JSX** — algo visual. Un custom hook **devuelve datos o funciones**, nunca JSX, y su nombre siempre empieza con use (convención obligatoria de React, no solo estilística: React usa ese prefijo para saber que ahí adentro se pueden llamar otros hooks).

useResponsiveLayout no dibuja nada en pantalla — es una función que **envuelve y reutiliza** la lógica de otro hook (useWindowDimensions, que viene de React Native) y le agrega valores calculados propios. Este patrón se llama “**hook compuesto**”: un hook que arma otros hooks para exponer una API más simple y específica al resto de la app.

2. useWindowDimensions() — el hook base de React Native

```
const { width, height } = useWindowDimensions();
```

Es un hook nativo de React Native (no hace falta instalarlo aparte) que devuelve el width/height actuales de la ventana de la app, y lo más importante: **se suscribe automáticamente a los cambios**. Si el usuario rota el dispositivo de portrait a landscape (o cambia el tamaño de la ventana, en web/desktop), useWindowDimensions dispara un re-render de cualquier componente que lo esté usando, con los nuevos valores. No hace falta escuchar manualmente el evento de rotación ni nada parecido — el hook se encarga.

3. Los valores calculados — derivados, no independientes

```
return {
  width,
  height,
  isLandscape: width > height,
  isWideScreen: width >= 700,
};
```

Ninguno de estos cuatro valores usa useState. Son **cálculos derivados** que se recalculan en cada render a partir de width/height — no hay estado propio del hook, todo depende pura y directamente de

lo que devuelve `useWindowDimensions()` en ese momento.

- **width/height:** se reexponen tal cual, por si alguna pantalla necesitara las dimensiones exactas en píxeles (en la práctica, ninguna pantalla de este proyecto las usa directamente — todas usan `isLandscape`).
- **isLandscape: width > height:** la definición de “modo horizontal” es simplemente comparar ancho contra alto. Si el ancho es mayor, el dispositivo está apaisado.
- **isWideScreen: width >= 700:** un umbral fijo (700 píxeles) para distinguir un teléfono de una tablet/pantalla ancha. Vale la pena mencionar que **isWideScreen está definido pero no se usa en ninguna pantalla actual del proyecto** (se comprobó buscando en todo `src/`) — está expuesto por el hook como una posibilidad para diseño responsive futuro (por ejemplo, mostrar un layout de dos columnas en tablets), pero hoy ninguna pantalla lo consume. Es un buen dato para mencionar si preguntan “¿usan esto?”: muestra que se entiende la diferencia entre lo que el hook ofrece y lo que la UI *actualmente aprovecha*.

4. Por qué es un hook y no una función común (`getIsLandscape()`)

Acá está el punto conceptual central: **tiene que ser un hook porque necesita re-ejecutarse en cada render cuando cambian las dimensiones**, y eso solo lo logra apoyándose en otro hook (`useWindowDimensions`) que internamente usa `useState/efectos` para suscribirse al evento nativo de cambio de dimensiones. Una función común (sin el prefijo `use`, llamada fuera del ciclo de render de React) no tendría forma de “engancharse” a esa reactividad — la calcularía una sola vez y quedaría desactualizada apenas el usuario rotara el dispositivo.

5. Dónde y cómo se usa — el mismo patrón en casi toda la app

Se importa en **12 archivos**: prácticamente todas las pantallas (`login.js`, `signup.js`, `home_user.js`, `mi_rutina_user.js`, `mi_cuenta_user.js`, `mi_cuenta_trainer.js`, `buscar_usuario_trainer.js`, `editar_rutina_trainer.js`, `ver_dia_x_user.js`, `ver_datos_trainer.js`, `reloj.js`), todas con el mismo patrón de uso:

```
const { isLandscape } = useResponsiveLayout();
// ...
<View style={[styles.card, isLandscape && styles.cardLandscape]}>
```

Cada pantalla desestructura solo lo que necesita (casi siempre únicamente `isLandscape`) y lo combina con un **array de estilos condicional**: el estilo base siempre se aplica, y el estilo `*Landscape` se agrega encima solo si `isLandscape` es `true`. Es el mismo mecanismo de estilos condicionales con `&&` que se ve en `login.js` (`cardLandscape`), `signup.js` (`scrollContentLandscape`), y en las barras de navegación (`bottomNavLandscape`).

También se le pasa `isLandscape` como **prop** a componentes hijos que lo necesitan pero no llaman el hook ellos mismos — por ejemplo `TrainerBottomNav/UserBottomNav`, que reciben `isLandscape` por `prop` en vez de invocar `useResponsiveLayout()` internamente. Esto es consistente con esos componentes: son “tontos” (reciben todo por props), así que la pantalla padre es quien llama al hook y les pasa el resultado.

Resumen para el oral

“useResponsiveLayout es un custom hook que envuelve useWindowDimensions de React Native, agregando dos valores derivados (isLandscape, isWideScreen) calculados a partir del ancho y alto

actuales de la ventana. Al ser un hook, se re-ejecuta automáticamente y dispara un re-render cada vez que el usuario rota el dispositivo, sin necesitar listeners manuales. Es consumido por prácticamente todas las pantallas de la app para aplicar estilos condicionales en modo horizontal, siguiendo siempre el mismo patrón: `style={{estiloBase, isLandscape && estiloLandscape}}.`”